

RALUZ COL S.A.S

System Architecture

Full-stack e-commerce platform for school uniforms

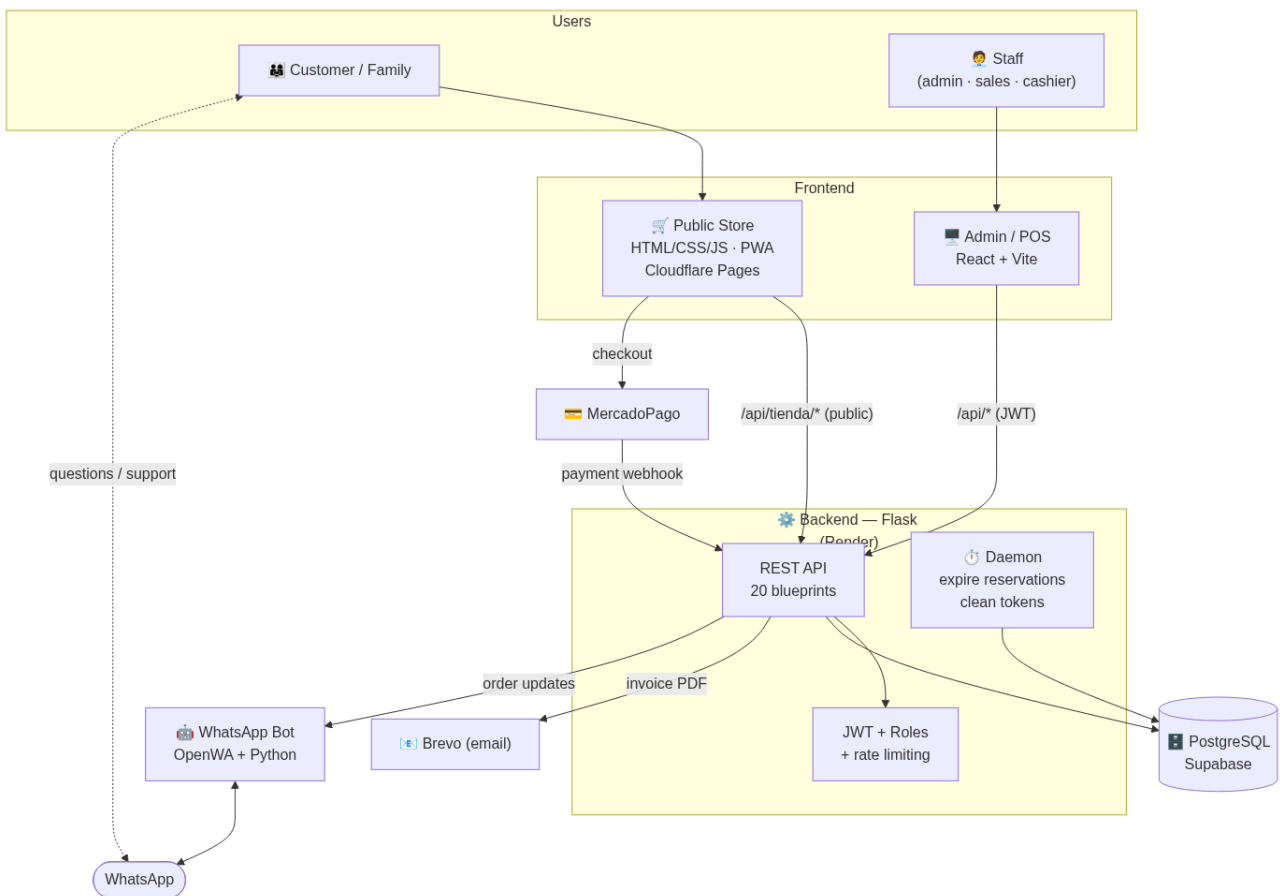
Metric	Value
Lines of code	~23,000
Backend (Flask)	8,000 LOC - 20 modules - 25 models
Admin panel (React)	9,200 LOC - 19 UI domains
Public store (JS)	6,000 LOC - offline PWA
Integrations	MercadoPago - Brevo - Google OAuth - WhatsApp
Deployment	Render - Cloudflare Pages - Supabase

What it is

An end-to-end system for a real school-uniform manufacturing and retail business in Bogota, Colombia. It covers the full lifecycle: catalog, online purchase, payment, invoicing, inventory, made-to-order production, delivery and support. It is made of three applications sharing a single backend: the public store (static site), the admin/POS panel (React SPA) and the backend (Flask REST API).

High-level architecture

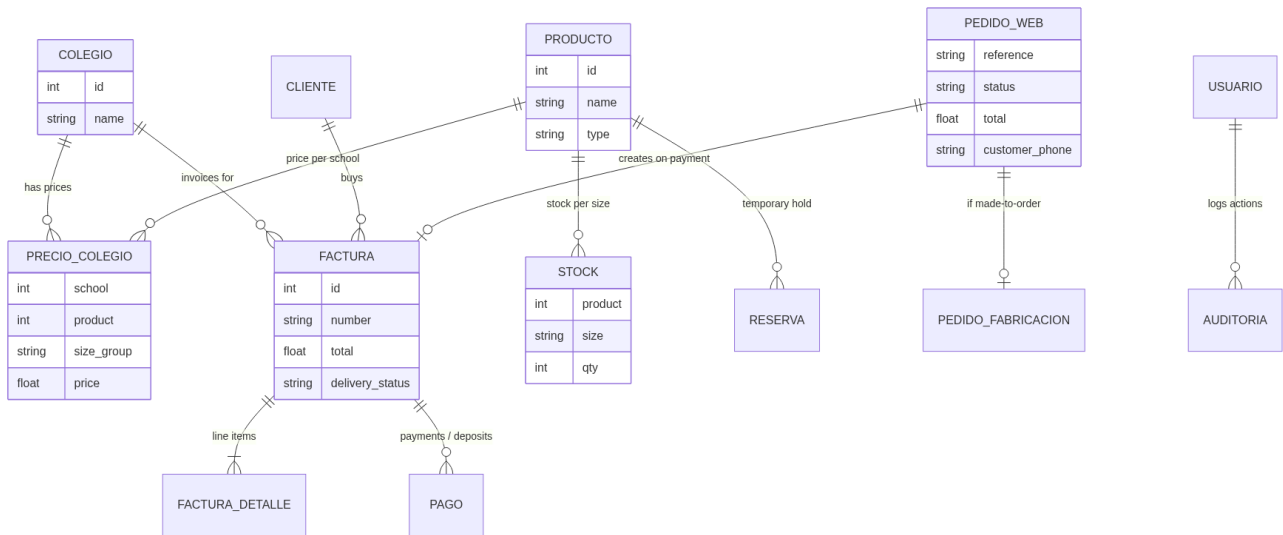
Three frontends and one central backend. The public store calls unauthenticated endpoints; the admin panel uses JWT. The backend integrates the database, payments, email and the WhatsApp bot.



High-level architecture

Data model

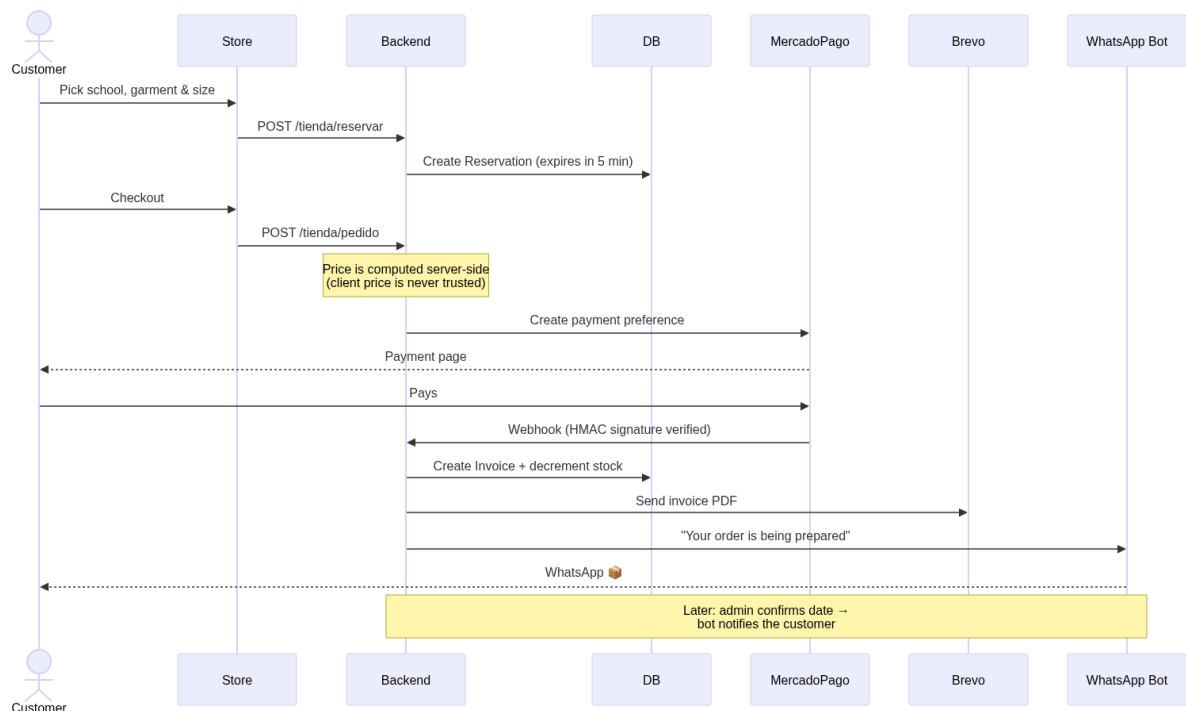
Core entities and their relationships. Prices are stored per size group and stock per individual size; a web order creates an invoice on payment and, if it needs production, a manufacturing order.



Data model

Core flow: online purchase

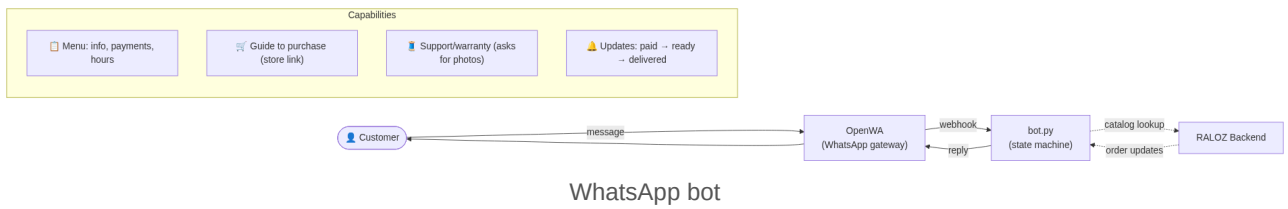
Price is computed server-side (the client price is never trusted). After payment, the MercadoPago webhook (HMAC-signed) creates the invoice, decrements stock, emails the PDF and notifies the customer over WhatsApp.



Core flow: online purchase

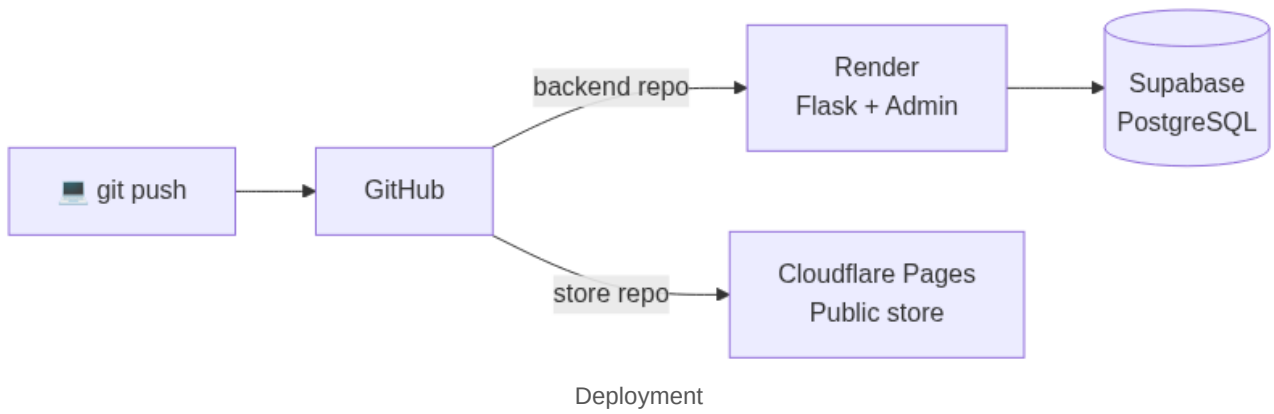
WhatsApp bot

Per-user stateful auto-responder (menu, support, warranty) plus automatic order-status updates. It reuses the web store for checkout instead of duplicating it.



Deployment

Continuous deployment from GitHub: backend and panel to Render, the store to Cloudflare Pages, and the database on Supabase.



Tech stack

Layer	Technology
Public store	HTML5, CSS3, JavaScript (ES Modules), Service Worker (offline PWA)
Admin panel	React 18, Vite, React Router, Axios (role-based protected routes)
Backend	Python, Flask, SQLAlchemy (REST API, one blueprint per domain)
Auth	Flask-JWT-Extended, bcrypt, Google OAuth, token blocklist
Database	PostgreSQL (Supabase), SSL
Payments	MercadoPago (preferences + HMAC-signed webhooks)
Email	Brevo API + ReportLab (in-memory PDF)
Infra	Render, Cloudflare Pages, Docker Compose

Security (audited and remediated)

- JWT authentication with account lockout and real logout (token blocklist).
- Role-based authorization; financial data is admin-only.
- Server-side price computation and HMAC-signed, idempotent payment webhook.
- Rate limiting, restricted CORS, security headers (HSTS, CSP) and SRI on CDNs.
- Anti-XSS (sanitization) and anti-SQL-injection (parameterized queries).
- Secrets only in environment variables and automated database backups.